The timer has to be reset every time a task starts to run. A typical real time tasks lasts for a few milliseconds. This leads to significant overheads as the call to reset a timer takes place every few milliseconds, which inturn leads to degraded performance. A more prevalent and better clock driven scheduling algorithm exists which we'll discuss next.

Cyclic scheduler

A large majority of small scale systems in the industry rely on cyclic schedulers. It's easy to program, maintain and is much more efficient as compared to table driven schedulers. A cyclic scheduler repeats a pre-computed schedule. The schedule constitutes of major cycles and which in turn contain minor cycles. The schedule needs to stored only for one major cycle, as all task repeat themselves in subsequent major cycles. A major cycle is divided into multiple minor cycles which are also known as frames. The scheduling point for a cyclic scheduler is the frame boundary i.e. a task can only be executed starting from a frame boundary. As this is clock driven scheduler, the frame boundaries are defined by clock interrupts. Each task is assigned to run in one or more frames. This assignment of a task to a frame is stored in a schedule table.

Interrupt driven

A fundamental drawback with clock driven schedulers is that firstly it becomes very complicated to generate a schedule for a large number of tasks and it's very difficult to arrive on an optimal frame size. This is where interrupt driven schedulers shine, they're much better at handling sporadic and aperiodic tasks. In an interrupt driven scheduler the scheduling points are defined by task arrival and task completion events. These schedulers are normally preemptive, which means that they stop a low priority task when a high priority task arrives. We'll discuss 3 important interrupt driven schedulers.

Simple priority based scheduling

Also known as foreground-background scheduler, it is the simplest priority based preemptive scheduler. There are only two priorities at which the tasks can run, the real time tasks run as foreground tasks or at high priority. The dynamically generated sporadic or aperiodic tasks are run as background tasks or at low priority. At a scheduling point, the foreground task is scheduled which is at a higher priority, the background task can only run if no foreground tasks are ready.